Avaliação — Grafos, Algoritmos de Busca e NP-Completude

PCC104 — Projeto e Análise de Algoritmos

DECOM — ICEB — UFOP

*Instruções: Esta prova é individual e possui **5 questões**. É permitido consultar o código que você mesmo implementou nas listas de exercícios. Justifique todas as respostas — respostas sem justificativa não recebem pontuação*

1. Busca em grafos.

Considere a função de busca genérica cujo trecho principal é reproduzido abaixo. Cada vértice é armazenado como `[id, cor, pai, dist]`, com cores `'B'` (branco), `'C'` (cinza) e `'P'` (preto).

```
while F:
    if type == 'dfs':
        x = F.pop(-1)
    elif type == 'bfs':
        x = F.pop(0)
    if x[0] in O:
        return x
    for xi in G[x[0]]:
        if V[xi][1] == 'B':
            V[xi][1] = 'C'
            V[xi][2] = x[0]
            V[xi][3] = V[x[0]][3] + 1
            F.append(V[xi])
    V[x[0]][1] = 'P'
```

(a) Explique qual estrutura de dados a lista `F` representa quando `type='dfs'` e quando `type='bfs'`, e por que essa única diferença (`pop(-1)` vs. `pop(0)`) altera a ordem de visitação.

(b) O campo `dist` (`V[xi][3]`) corresponde ao número de arestas no caminho a partir da origem. Esse valor é garantidamente o *menor* número de arestas até cada vértice na versão `bfs`? E na versão `dfs`? Justifique.

2. Representação de grafos.

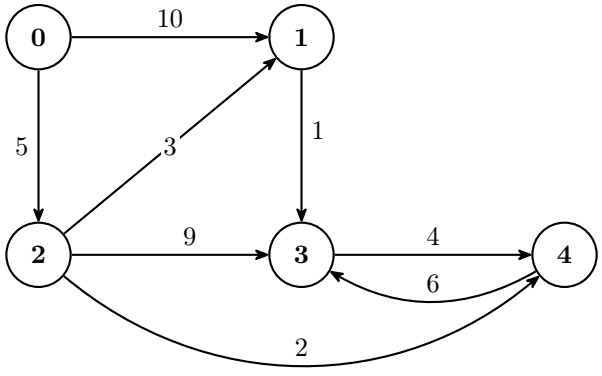
Considere um grafo com n vértices representado por uma matriz de adjacência M de dimensão $n \times n$.

(a) Descreva, em pseudocódigo ou em texto preciso, como implementar as operações `inserir_aresta(u,v)` e `remover_vertice(key)`. Indique a complexidade de cada uma em função de n.

(b) A mesma operação `remover_vertice(key)`, se o grafo estivesse representado por lista de adjacência, teria complexidade diferente? Compare os dois casos justificando.

3. Caminhos mínimos (Dijkstra).

Considere o grafo direcionado ponderado abaixo, com origem no vértice 0.



(a) Execute o algoritmo de Dijkstra a partir de 0. Apresente uma tabela mostrando, a cada extração de vértice, a distância `d` e o predecessor `pai` de cada vértice. Indique a distância final até todos os vértices.

(b) Na sua implementação (`dijkstra.py`), a atualização só ocorre quando `v['d'] > u['d'] + w[(u,v)]`. Explique o que essa condição representa e por que ela é essencial para a corretude do algoritmo.

4. NP-Completude.
Um estudante quer provar que Vertex-Cover é NP-difícil e afirma: *"Vou reduzir Vertex-Cover a Clique, que já sabemos ser NP-completo, e isso prova que Vertex-Cover é NP-difícil."* O argumento está correto? Aponte o erro (se houver) e indique qual é a direção correta da redução entre Clique e Vertex-Cover.

5. Vivendo com problemas NP-completos.
Uma empresa de logística precisa resolver, no dia a dia, instâncias de um problema NP-completo de roteamento. Diante da impossibilidade de uma garantia polinomial de pior caso, a equipe de engenharia adota abordagens diferentes conforme o contexto. Para cada situação abaixo, identifique qual das cinco estratégias da Seção 8 das notas está sendo empregada (algoritmo exponencial rápido na prática, caso especial tratável, algoritmo de aproximação, heurística/metaheurística, ou complexidade parametrizada) e justifique em uma frase.

(a) Para o planejamento noturno da frota inteira, a equipe roda um solver de programação por restrições com *backtracking* e poda agressiva; ele quase sempre termina em minutos, embora não haja garantia de tempo no pior caso.

(b) Percebendo que as entregas de uma certa região sempre acontecem ao longo de uma única via expressa (o grafo de rotas é essencialmente um caminho), a equipe usa um algoritmo específico que resolve esse formato em tempo polinomial.

(c) Para um relatório rápido de estimativa de custo, basta uma solução comprovadamente no máximo o dobro do ótimo, obtida por um procedimento garantidamente polinomial.

(d) Um módulo do sistema precisa alocar apenas até 5 veículos especiais por vez; a equipe usa um algoritmo cujo tempo é exponencial somente nesse número pequeno de veículos, mas linear no tamanho total do mapa.

(e) Sem exigir nenhuma garantia formal de qualidade, a equipe aplica *simulated annealing* para melhorar iterativamente as rotas de um turno, aceitando o resultado que estiver disponível quando o tempo acabar.

Boa prova!